

# DATA PREPROCESSING PIPELINE

*Подробное руководство пользователя*

© Андрей Трегубов, 2026.

## Содержание

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| 1. Введение .....                                                            | 4         |
| 1.1 Назначение программы .....                                               | 4         |
| 1.2 Входные данные .....                                                     | 4         |
| 1.2.1 Формат файлов данных .....                                             | 4         |
| 1.2.2 Информационный файл .INFO .....                                        | 5         |
| 1.2.3 Загрузка файлов в программу .....                                      | 5         |
| 1.3 Общий поток обработки данных .....                                       | 6         |
| 1.4 Обнаружение и удаление погружений (дайв-детектор).....                   | 6         |
| 1.4.1 Проблема.....                                                          | 6         |
| 1.4.2 Алгоритм обнаружения .....                                             | 6         |
| 1.4.4 Визуализация и ручная коррекция .....                                  | 7         |
| 1.5 Субсемплинг и кнопка «Построить полностью» .....                         | 7         |
| 1.5.1 Субсемплированный кеш для интерактивной работы .....                   | 8         |
| 1.5.2 Кнопка «Построить полностью» .....                                     | 8         |
| 1.6 Преобразования Фурье: полное описание .....                              | 8         |
| 1.6.1 Зачем нужна фильтрация Фурье .....                                     | 8         |
| 1.6.2 Прямое преобразование Фурье (Шаг 3, вычисление спектра) .....          | 8         |
| 1.6.3 Визуализация спектра .....                                             | 9         |
| 1.6.4 Выбор частоты среза .....                                              | 9         |
| 1.6.5 Зеркальное дополнение (Mirror Padding) и обратное преобразование ..... | 9         |
| 1.6.6 Окно перед/после фильтрации.....                                       | 9         |
| 1.7 Проход по массиву и вычисление волновых параметров .....                 | 10        |
| 1.7.1 Структура прохода .....                                                | 10        |
| 1.7.2 RMS-фильтрация .....                                                   | 10        |
| 1.7.3 Удаление спайков .....                                                 | 10        |
| 1.7.4 Сплайн-интерполяция .....                                              | 11        |
| 1.7.5 Среднеквадратичное отклонение, $A_s$ и $H_s$ .....                     | 11        |
| 1.7.6 Индивидуальные амплитуды, высоты волн, $T_z$ .....                     | 11        |
| 1.7.7 $A_{s\_1/3}$ и $H_{s\_1/3}$ — параметры 1/3 наибольших волн.....       | 12        |
| 1.7.8 Решение дисперсионного уравнения ( $k_h$ , $k$ ) .....                 | 12        |
| 1.7.9 Геометрические и нелинейные параметры .....                            | 12        |
| 1.7.10 Спектральные параметры ( $Q$ , $v$ , $\epsilon_w$ ).....              | 13        |
| 1.7.11 Параметр регулярности $\rho$ .....                                    | 13        |
| 1.7.12 Индекс Бенджамина–Фейра (BFI).....                                    | 14        |
| 1.8 Выходные данные программы .....                                          | 14        |
| <b>2. Пошаговое руководство пользователя.....</b>                            | <b>16</b> |
| <b>2.1 Подготовка к работе.....</b>                                          | <b>16</b> |
| <b>2.1.1 Системные требования .....</b>                                      | <b>16</b> |

|                                                  |    |
|--------------------------------------------------|----|
| 2.1.2 Запуск программы .....                     | 16 |
| 2.1.3 Папка Output и система checkpoint'ов ..... | 16 |
| 2.2 Шаг 1 — Загрузка и конвертация файлов .....  | 16 |
| 2.2.1 Добавление файлов .....                    | 16 |
| 2.2.2 Запуск конвертации .....                   | 17 |
| 2.2.3 Итоговое сообщение .....                   | 17 |
| 2.3 Шаг 2 — Визуализация и обрезка .....         | 18 |
| 2.3.1 Чтение графика .....                       | 18 |
| 2.3.2 Ручная обрезка .....                       | 19 |
| 2.3.3 Что происходит с данными .....             | 20 |
| 2.4 Шаг 3 — Фурье-фильтрация .....               | 20 |
| 2.4.1 Вычисление спектра .....                   | 20 |
| 2.4.2 Выбор частоты среза .....                  | 21 |
| 2.4.3 Применение фильтра и окно сравнения .....  | 21 |
| 2.5 Шаг 4 — Анализ и вычисление параметров ..... | 22 |
| 2.5.1 Настройки перед запуском .....             | 22 |
| 2.5.2 Процесс обработки в реальном времени ..... | 22 |
| 2.5.3 Окно Pipeline Complete .....               | 23 |
| 2.6 Экспорт результатов .....                    | 24 |
| 2.6.1 Загрузка результатов в MATLAB .....        | 24 |
| 2.6.2 Загрузка результатов в Python .....        | 25 |

# 1. Введение

## 1.1 Назначение программы

Data Preprocessing Pipeline — это десктопное приложение с графическим интерфейсом, предназначенное для **полного цикла обработки данных придонных датчиков давления**. Программа принимает на вход «сырые» записи — высоту (в метрах) водяного столба, последовательно очищает их от артефактов, применяет фильтрацию Фурье, и вычисляет набор волновых параметров.

Программа полностью автоматизирует следующие задачи, которые традиционно выполняются вручную в MATLAB или Python:

- Конвертация бинарных .dat файлов в единый временной ряд с метками времени, разбиение их на 20-минутные статистически однородные фрагменты
- Визуальная инспекция и ручное обрезание данных по началу и концу записи
- Удаление тренда и вычисление нуля-среднего сигнала смещения поверхности
- Вычисление полного спектра Фурье и интерактивный выбор частоты среза
- Обратное преобразование Фурье с зеркальным дополнением для подавления краевых артефактов (эффект Гиббса)
- Автоматическое удаление низкоэнергетических записей и спайков
- Расчёт 20+ волновых параметров для каждой записи
- Экспорт массивов индивидуальных амплитуд и высот волн

## 1.2 Входные данные

### 1.2.1 Формат файлов данных

Программа принимает на вход **набор текстовых файлов .dat** и один **информационный файл .INFO**. Файлы .dat содержат по одному числу на строку. Каждое число — это **высота водного столба (расстояние между датчиком и поверхностью воды), в метрах**. Физически это значение позволяет вычислить глубину установки датчика и получить смещение свободной поверхности.

**⚠ Примечание:** Если исходные данные датчика записаны в единицах давления (Па/мм Рт.Ст.), необходимо выполнить гидростатический пересчёт в метры водяного столба ДО загрузки в программу.

Пример содержимого файла .dat:

```
8.294242314722688 ← длина водного столба, м (точка)
8.276029385536757
8,359068152086337 ← запятая тоже допустима
8.534720557960874
```

Десятичный разделитель может быть **точкой или запятой** — программа распознаёт оба варианта автоматически. Строки, которые не удаётся распознать как число, молча пропускаются — заголовки и текстовые комментарии в файле не мешают загрузке.

Каждый файл .dat соответствует непрерывному временному блоку записи. Датчик генерирует **равномерно дискретизированный** сигнал с постоянной частотой дискретизации (например, 1 Гц или 8 Гц). Файлов может быть несколько — программа объединяет их в хронологическом порядке в единый массив.

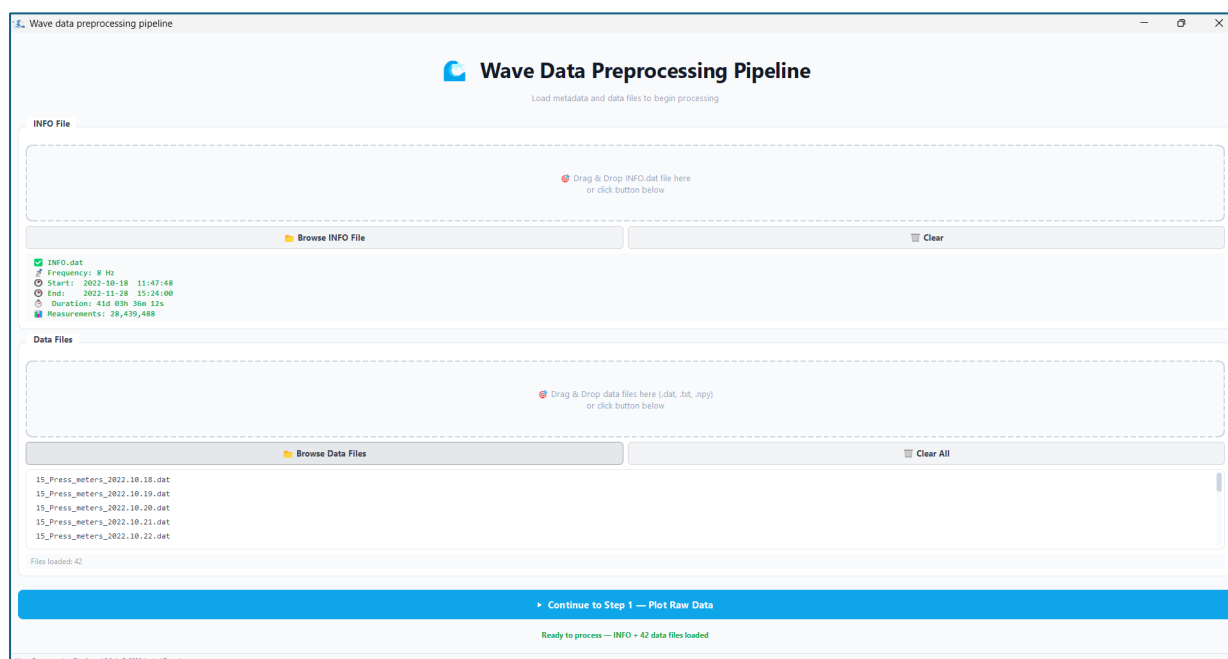


Рис. 1 — окно загрузки с добавленными файлами .dat и INFO

## 1.2.2 Информационный файл .INFO

Информационный файл .INFO — это текстовый файл произвольной структуры, содержащий метаданные записи. Программа читает его с помощью **регулярных выражений**, поэтому **порядок полей в файле не важен** — нужные данные будут найдены в любом месте документа. Файл может содержать произвольный текст, комментарии и дополнительные поля — всё лишнее игнорируется.

Программа извлекает из .INFO файла следующие обязательные данные:

- **Частота дискретизации датчика.** Ожидается строка вида `Частота опроса: 8 Гц` или `Frequency: 8 Hz`. Если частота не найдена — загрузка завершается с ошибкой.
- **Дата и время начала записи.** Ожидается строка с датой и временем в формате `ГГГГ.ММ.ДД ЧЧ:ММ:СС`, например `2022.10.18 11:47:48.000`. Первая найденная дата считается началом записи.
- **Дата и время окончания записи.** Вторая дата в том же формате. Используется только для проверки целостности данных.

Файл может быть записан в кодировке UTF-8, Windows-1251 (cp1251) или Latin-1 — программа перебирает все варианты автоматически. Глубина установки датчика **не читается из .INFO файла**: она вычисляется из самих данных.

## 1.2.3 Загрузка файлов в программу

Для загрузки данных необходимо:

1. **Запустить программу** — откроется главное окно Шага 1 (Load & Convert).

2. Нажать кнопку **«Добавить файлы»** или перетащить файлы .dat и .INFO прямо в окно программы (drag & drop).
3. Убедиться, что в списке присутствует ровно один файл .INFO и нужные файлы .dat.
4. Нажать **«Process & Convert»**. Программа объединит все .dat файлы в единый массив, сгенерирует временные метки от даты начала из .INFO и разобьёт ряд на 20-минутные блоки.

**⚠ Примечание:** Если сгенерированная временная метка последней точки данных отличается от ожидаемого времени окончания записи из .INFO более чем на 1 час (допустимая погрешность прибора для записей длительностью > 1 месяца), программа покажет предупреждение с указанием фактического и ожидаемого времени. Это может означать неполный набор .dat файлов или ошибку в .INFO файле.

## 1.3 Общий поток обработки данных

Программа организована в виде последовательного пятишагового конвейера. Каждый шаг сохраняет результат в папку Output в виде CSV-файла, что позволяет при необходимости возобновить обработку с любого шага без повторного выполнения предыдущих.

| Шаг   | Название        | Входной файл          | Выходной файл(ы)                         |
|-------|-----------------|-----------------------|------------------------------------------|
| Шаг 1 | Load & Convert  | .dat + .INFO          | Step1_TXTtoCSV.csv                       |
| Шаг 2 | Visualize & Cut | Step1_TXTtoCSV.csv    | Step2_Zero_Mean.csv                      |
| Шаг 3 | Fourier Filter  | Step2_Zero_Mean.csv   | Step3_Transformed.csv                    |
| Шаг 4 | Analyze         | Step3_Transformed.csv | Parameters.csv<br>Amplitudes_Heights.csv |

## 1.4 Обнаружение и удаление погружений (дайв-детектор)

### 1.4.1 Проблема

При развёртке и подъёме придонного датчика в начале и конце записи неизбежно присутствуют **«ноги погружения»** (dive legs) — переходные участки, когда датчик движется через водную толщу от поверхности ко дну и обратно. В эти моменты сигнал давления представляет собой **монотонно нарастающий или убывающий тренд**, а не волновое движение. Эти участки необходимо идентифицировать и исключить до любого анализа волн.

### 1.4.2 Алгоритм обнаружения

Программа использует **анализ градиента сигнала** для автоматического обнаружения ног погружения. Алгоритм работает следующим образом:

**Шаг 1. Вычисление градиента и порога:**

```
gradient[i] = (y[i+1] - y[i-1]) / 2
```

*численный градиент (центральные разности)*

```
threshold = sensitivity * std(gradient)
```

*где sensitivity = 3.0 по умолчанию*

### Шаг 2. Нога начала записи (поиск в первых 40% данных):

Программа ищет все моменты резкого положительного скачка градиента, где одновременно выполняются условия:

- `gradient[i] > threshold` — значительный скачок
- `y[i-1] < 2.0 м` — датчик был у поверхности/в воздухе до скачка
- `y[i+1] ≥ 2.0 м` — датчик оказался на глубине после скачка

Граница ноги устанавливается по **последнему** такому скачку (датчик мог несколько раз погружаться и всплывать перед финальной установкой). После последнего скачка программа движется вперёд до тех пор, пока градиент не успокоится ниже `grad_std`. Дополнительно добавляется 10% запас. Весь участок от начала до этой точки помечается как нога погружения и исключается.

### Шаг 3. Нога конца записи (поиск в последних 40% данных):

Аналогично, но ищутся резкие отрицательные скачки:

- `gradient[i] < -threshold`
- `y[i-1] ≥ 2.0 м` — датчик был на глубине
- `y[i+1] < 2.0 м` — датчик покинул воду

Граница устанавливается по **первому** такому событию. Ходим назад до успокоения градиента, добавляем 10% запас. Весь хвост помечается как нога подъёма.

#### 1.4.4 Визуализация и ручная коррекция

После автоматического обнаружения программа выделяет ноги погружения **красным цветом** на графике. Если детектор сработал, становится активной кнопка **«Ручная обрезка»**, которая позволяет вручную скорректировать границы: пользователь кликает на графике для указания начала и/или конца рабочего участка записи.

**Если погружений не обнаружено** — кнопка ручной коррекции остаётся неактивной. Это нормальная ситуация, если запись уже предварительно обрезана или датчик был стационарно закреплён на нужной глубине с самого начала.

## 1.5 Субсемплинг и кнопка «Построить полностью»

Полные записи могут содержать **сотни тысяч и миллионы точек** (например, 30 суток при 8 Гц = ~20 миллионов точек). Отображение такого количества точек в matplotlib в реальном времени невозможно — это заморозило бы интерфейс на десятки секунд при каждом масштабировании.

Программа решает эту проблему двухуровневым подходом:

### 1.5.1 Субсемплированный кеш для интерактивной работы

При обработке каждого шага программа автоматически создаёт **субсемплированный визуализационный кеш** — прореженную версию данных, содержащую не более 5 000–10 000 точек (чуть более для Фурье-спектров).

Этот кеш используется для всех интерактивных операций — масштабирования, панорамирования, выбора точек обрезки. Визуально разница между субсемплом и полными данными при обзорном масштабе практически незаметна.

### 1.5.2 Кнопка «Построить полностью»

На каждом шаге в панели инструментов графика есть кнопка **«Построить полностью»** (или «Build all data points»). При её нажатии программа загружает полные данные из CSV и перестраивает график без прореживания. Это **медленная операция** (может занять несколько секунд), но позволяет увидеть точную форму сигнала — особенно важно при работе с Шагом 2 (обрезка начала/конца) и Шагом 3 (выбор частоты среза).

**⚠ Примечание:** При работе с кнопками обрезки (установка начала и конца записи), если важна максимальная точность, рекомендуется нажать «Построить полностью» и только затем кликать на графике для выбора точки обрезки — так отметка будет соответствовать реальному положению в данных, а не субсемплированной версии.

## 1.6 Преобразования Фурье: полное описание

### 1.6.1 Зачем нужна фильтрация Фурье

После вычисления нуль-среднего сигнала смещения поверхности  $\eta(t)$  в нём всё ещё присутствуют **низкочастотные компоненты** — приливные вариации, инфрагравитационные волны, медленные дрейфы. Эти компоненты имеют периоды сотни секунд и более и не относятся к ветровым волнам. Для корректного вычисления волновых параметров их необходимо удалить.

Программа применяет **высокочастотный проход в частотной области**: обнуляет все компоненты спектра ниже выбранной пользователем частоты среза  $\omega_c$ . Физически это эквивалентно удалению из сигнала всех волн с периодом больше чем  $T_c = 2\pi/\omega_c$ .

### 1.6.2 Прямое преобразование Фурье (Шаг 3, вычисление спектра)

Программа применяет к отфильтрованному нуль-среднему сигналу **быстрое преобразование Фурье (БПФ)** — алгоритм, который разлагает временной ряд на набор синусоид разных частот. Каждой частоте ставится в соответствие амплитуда и фаза. В результате получается **комплексный спектр**: массив комплексных чисел, по одному на каждую частоту.

Частоты выражаются в **угловых единицах (рад/с)**. Весь спектр, включая вещественную и мнимую части, сохраняется в `Step3_Spectrum.csv` — это необходимо для точного восстановления сигнала при обратном преобразовании.

**⚠ Примечание:** Рекомендуется выбирать частоту среза так, чтобы удалять компоненты с периодом больше 10 минут ( $\omega < 0.0105$  рад/с). Более высокочастотный срез трогать не нужно — ветровые волны находятся значительно правее.



### 1.6.3 Визуализация спектра

Спектр отображается в двух окнах одновременно:

- **Верхний график** — полный спектр для  $\omega > 0.1$  рад/с (линейная шкала). Отображает ветровые волны.
- **Нижний график** — диапазон  $0 < \omega \leq 0.1$  рад/с (логарифмическая шкала по оси  $\omega$ ). Отображает низкочастотные компоненты — приливы, инфрагравитационные волны.

### 1.6.4 Выбор частоты среза

Пользователь выбирает **частоту среза**  $\omega_c$  двойным кликом по нижнему графику. После клика программа:

5. Рисует вертикальную красную линию на выбранной частоте
6. Отображает соответствующий период:  $T_c = 2\pi / \omega_c$  (в секундах)
7. Закрашивает область левее линии — компоненты, которые будут удалены
8. Активирует кнопку «Apply & Continue» для перехода к обратному преобразованию

### 1.6.5 Зеркальное дополнение (Mirror Padding) и обратное преобразование

Прямое обнуление низких частот в спектре эквивалентно идеальной прямоугольной фильтрации. Прямоугольное окно в частотной области порождает **эффект Гиббса** в временной области — осцилляции и «звон» вблизи краёв сигнала (первые и последние несколько периодов). Для подавления этого артефакта программа применяет **зеркальное дополнение (mirror padding)**.

#### Идея метода

Перед фильтрацией к сигналу «пристыковываются» специальные **зеркальные хвосты** с обеих сторон. Левый хвост — это начало сигнала, отражённое в обратном порядке. Правый хвост — это конец сигнала, тоже отражённый. В результате сигнал становится длиннее, а его края оказываются **плавно продолженными** без разрывов.

Длина каждого хвоста выбирается равной одному полному периоду частоты среза — то есть ровно такому участку, на протяжении которого затухает краевой артефакт. Максимальная длина хвоста ограничена четвертью длины исходного сигнала, чтобы не раздуть массив слишком сильно.

#### Этапы обработки:

1. К сигналу пристыковываются зеркальные хвосты с обеих сторон.
2. К расширенному сигналу применяется БПФ.
3. В спектре обнуляются все компоненты с частотой ниже выбранного среза.
4. Выполняется обратное БПФ — получается отфильтрованный расширенный сигнал.
5. Хвосты отбрасываются. Они уже «поглотили» краевые осцилляции. Возвращается отфильтрованный сигнал исходной длины — без артефактов на краях.

### 1.6.6 Окно перед/после фильтрации

После обратного преобразования программа показывает **окно сравнения «Before / After»**. На одном графике **оранжевым цветом** отображается исходный нуль-средний

сигнал до фильтрации, **синим** — отфильтрованный сигнал. Это позволяет убедиться, что фильтрация корректна: долгопериодный тренд удалён, форма ветровых волн сохранена.

## 1.7 Проход по массиву и вычисление волновых параметров

### 1.7.1 Структура прохода

Шаг 4 работает с отфильтрованным сигналом смещения поверхности. Программа последовательно проходит по каждому 20-минутному фрагменту и для каждого вычисляет набор параметров.

Для каждой записи выполняются следующие этапы:

- **Шаг 1:** RMS-фильтрация (опционально)
- **Шаг 2:** Удаление спайков (опционально)
- **Шаг 3:** Подготовка массива (опциональная сплайн-интерполяция)
- **Шаг 4:** Вычисление амплитудных параметров ( $\sigma$ ,  $A_s$ ,  $H_s$ ,  $A_{s\_1/3}$ ,  $H_{s\_1/3}$ )
- **Шаг 5:** Извлечение нуль-пересечений, амплитуд, высот и  $T_z$
- **Шаг 6:** Решение дисперсионного уравнения ( $k_h$ ,  $k$ )
- **Шаг 7:** Геометрические и нелинейные параметры ( $\epsilon$ ,  $a$ ,  $U_r$ )
- **Шаг 8:** Спектральные параметры ( $Q$ ,  $v$ ,  $\epsilon_w$ )
- **Шаг 9:** Параметр регулярности  $\rho$  ( $\rho$ )
- **Шаг 10:** Производные параметры ( $\epsilon_\rho$ ,  $\gamma$ ,  $BFI$ )

### 1.7.2 RMS-фильтрация

Если включён чекбокс **«Remove recordings with RMS <»**, для каждой записи вычисляется среднеквадратическое отклонение (RMS) сигнала:

$$RMS = \sqrt{\text{mean}(\eta^2)}$$

*среднеквадратическое значение смещения поверхности*

Если  $RMS < \text{threshold}$  (по умолчанию 0.015 м), запись считается «тихой» — энергия волнения слишком мала для надёжного анализа, скорее всего на поверхности лежит лёд. Запись **исключается из дальнейших расчётов** и помечается красным цветом на графике. Все последующие шаги для неё не выполняются.

### 1.7.3 Удаление спайков

Если включён чекбокс **«Remove spikes»**, программа ищет в записи резкие скачки амплитуды, характерные для электронных помех датчика. Порог определяется через дисперсию:

$$\text{threshold\_spike} = 6 \cdot \sqrt{\text{Var}(\eta)}$$

*6 стандартных отклонений = спайк*

После обнаружения спайк заменяется на среднее арифметическое от соседей. Каждый обнаруженный спайк **отмечается кружком** на графике (сам спайк может не попасть в субсемплированные данные).

#### 1.7.4 Сплайн-интерполяция

Если включён чекбокс **«Spline interpolation»** и целевая частота  $\text{target\_hz} > \text{sensor\_freq}$ , программа строит кубический сплайн на исходном сигнале и пересемплирует его на более высокую частоту:

```
arr_amp = CubicSpline(t_orig, arr)(t_new)
где t_new = 0, 1/target_hz, 2/target_hz, ...
```

Результат `arr_amp` используется для вычисления амплитудных параметров. Интерполяция повышает точность оценки максимума в каждой полуволне, не добавляя новой спектральной информации. Все спектральные параметры ( $Q$ ,  $v$ ,  $\varepsilon$ ) вычисляются по **исходному** массиву.

#### 1.7.5 Среднеквадратичное отклонение, $A_s$ и $H_s$

Стандартное отклонение вычисляется по интерполированному массиву `arr_amp`:

$$\sigma = \sqrt{\text{Var}(\text{arr\_amp})}$$

Значительная амплитуда:

$$A_s = 2\sigma$$

Значительная высота волн:

$$H_s = 4\sigma$$

#### 1.7.6 Индивидуальные амплитуды, высоты волн, $T_z$

Программа находит все пересечения нуля и выделяет набор индивидуальных волн (точки между двумя соседними пересечениями нуля). Затем вычисляются:

**Амплитуда** каждой полуволны:

$$A_i = \max(|\text{arr\_amp}[\text{xc}[i] : \text{xc}[i+1]]|)$$

*максимальное абсолютное смещение в сегменте*

**Высота** каждой волны (сумма двух соседних амплитуд — гребень + впадина):

$$H_i = A_i + A_{\{i+1\}}$$

*$i$ -я волна образована  $i$ -й и  $(i+1)$ -й полуволнами*

**Средний период** нуль-пересечений  $T_z$ :

$$T_z = (N / f_s) / N_{\text{crossings}} \times 2$$

$N_{\text{crossings}}$  — количество нуль-пересечений; делитель 2 — переход от полупериодов к периодам

### 1.7.7 $As_{1/3}$ и $Hs_{1/3}$ — параметры 1/3 наибольших волн

Стандартные океанографические параметры значительной амплитуды и высоты, вычисленные напрямую из статистики индивидуальных волн (а не через  $\sigma$ ):

$$As_{1/3} = \text{mean}(\text{sorted}(A_i, \text{desc})[ : N/3 ])$$

среднее арифметическое трети наибольших амплитуд

$$Hs_{1/3} = \text{mean}(\text{sorted}(H_i, \text{desc})[ : N/3 ]) \quad \text{среднее арифметическое трети наибольших высот волн}$$

### 1.7.8 Решение дисперсионного уравнения ( $kh$ , $k$ )

Для мелководья и промежуточных глубин связь между периодом волны и волновым числом описывается **уравнением дисперсии линейной теории волн**:

$$\omega^2 = g \cdot k \cdot \tanh(kh)$$

где  $\omega = 2\pi/T_z$  — угловая частота,  $g = 9.81 \text{ м/с}^2$ ,  $h$  — глубина,  $k$  — волновое число

Введём безразмерный параметр  $kh$ . Тогда уравнение переписется в форме:

$$kh \cdot \tanh(kh) = 4\pi^2 h / (g \cdot T_z^2)$$

нелинейное уравнение относительно  $kh$ , решается численно

Программа решает это уравнение методом метод Ньютона–Рафсона. Начальное приближение выбирается адаптивно:

- $x_0 = \text{target}$  если  $\text{target} < 1$  (мелководье)
- $x_0 = \sqrt{\text{target}}$  если  $1 \leq \text{target} < 10$  (промежуточная глубина)
- $x_0 = \text{target}$  если  $\text{target} \geq 10$  (глубоководье)

После нахождения  $kh$  вычисляется волновое число:

$$k = kh / h$$

### 1.7.9 Геометрические и нелинейные параметры

**Крутизна волны  $\varepsilon$ :**

$$\varepsilon = (k \cdot H_s) / 4$$

безразмерный параметр;  $\varepsilon \rightarrow 1/7$  соответствует волнам Стокса на пороге опрокидывания

**Относительная амплитуда  $a$ :**

$$a = A_s / h$$

отношение амплитуды к глубине

Число Урселла  $U_r$ :

$$U_r = (3 \cdot k \cdot H_s) / (2 \cdot k \cdot h)^3$$

число, отвечающее за баланс дисперсии и нелинейности

### 1.7.10 Спектральные параметры ( $Q$ , $\nu$ , $\varepsilon_w$ )

Функция `calculate_spectrum_params` вычисляет параметры из спектральных моментов. Сначала применяется **окно Ханна** для уменьшения краевых эффектов:

$$S(\omega) = |\text{FFT}(\text{arr} \times w_{\text{Hann}})|$$

где  $w_{\text{Hann}}[n] = 0.5 \cdot (1 - \cos(2\pi n / (N-1)))$

Вычисляются спектральные моменты на «адекватной» части спектра, для всех  $\omega$ , меньших, чем **Sensor\_Frequency** \*  $\pi/2$  («хвост» спектра всегда шумный и «забывает» момент 2-го и 4-го порядка):

$$m_n = \int \omega^n \cdot S(\omega) d\omega$$

Параметр Годы  $Q$ :

$$Q = \int \omega \cdot S^2(\omega) d\omega / m_0^2$$

Параметр ширины спектра  $\nu$ :

$$\nu = \sqrt{(m_0 \cdot m_2 / m_1^2 - 1)}$$

Параметр ширины спектра  $\varepsilon_w$ :

$$\varepsilon_w = \sqrt{(1 - m_2^2 / (m_0 \cdot m_4))}$$

### 1.7.11 Параметр регулярности $\rho$

Параметр  $\rho$  характеризует насколько каждая индивидуальная волна «правильная» — имеет один максимум. Для его вычисления так же как и для спектральных моментов применяется **дополнительная фильтрация в хвосте спектра** для подавления шума (аналогично, вырезается хвост до  $\omega = \text{Sensor\_Frequency} \cdot \pi/2$ ).

$$\rho = N_{\text{waves}} / N_{\text{extrema}}$$

$N_{\text{waves}}$  — количество индивидуальных волн;  $N_{\text{extrema}}$  — суммарное число экстремумов

Производная ширина спектра через  $\rho$ :

$$\varepsilon_{\rho} = 2\sqrt{(1 - \rho) / (2 - \rho)}$$

альтернативная мера ширины спектра;  $\varepsilon_{\rho} = 0$  при  $\rho = 1$

**⚠ Примечание:** Параметр ширины спектра  $\varepsilon$  теоретически не должен зависеть от метода его измерения, однако на практике действительно широкие спектры для точного измерения их ширины методом нахождения отношения числа индивидуальных волн к общему числу экстремумов требуют очень чувствительных сенсоров (>10 Гц), и, вероятно, более длительных отрезков времени (>20 мин) – **нужны дополнительные численные эксперименты для сравнения точности измерений этими двумя методами на модельных данных**. Для данных менее 8 Гц измерение параметра регулярности почти всегда некорректно, т.к. экстремумы просто не «прописываются» и лучше вычислять спектральные моменты. Для данных >10 Гц наоборот рекомендуется использовать параметр регулярности, т.к. инструментальный «шум» хвостов спектра завышает значение спектральных моментов.

### 1.7.12 Индекс Бенджамина–Фейра (BFI)

Индекс Бенджамина–Фейра (Benjamin–Feir Index)

**BFI\_proper** (полная формула):

$$\text{BFI\_proper} = \sqrt{(2\pi)} \cdot \varepsilon \cdot Q \cdot \gamma$$

$\varepsilon$  — крутизна,  $Q$  — параметр Годы,  $\gamma$  — нелинейный коэффициент

**BFI\_goda** (упрощённая формула Годы):

$$\text{BFI\_goda} = \sqrt{(2\pi)} \cdot \varepsilon \cdot Q$$

без нелинейной поправки  $\gamma$

**Нелинейный коэффициент  $\gamma$**  (функция  $kh$ ):

$$v = 1 + 2kh / \sinh(2kh)$$

$$a = -v^2 + 2 + 8(kh)^2 \cdot \cosh(2kh) / \sinh^2(2kh)$$

$$b = [\cosh(4kh) + 8 - 2\tanh^2(kh)] / [8\sinh^4(kh)] - [2\cosh^2(kh) + v/2]^2 / [\sinh^2(2kh) \cdot (kh/\tanh(kh) - v^2/4)]$$

$$\gamma = v \cdot \sqrt{(|b| / |a|)}$$

## 1.8 Выходные данные программы

По завершении Шага 4 программа создаёт следующие файлы в папке Output:

| Файл                                | Содержимое                                                                                                                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Parameters.csv</code>         | Все волновые параметры для каждой 20-минутной записи: RMS, As, Hs, As_1/3, Hs_1/3, Tz, kh, k, $\epsilon$ , a, Ur, Q, v, $\epsilon_w$ , $\rho$ , $\epsilon_p$ , $\gamma$ , BFI_proper, BFI_goda |
| <code>Step4_Filtered.csv</code>     | Временной ряд смещения поверхности после удаления низкоэнергетических записей и исправления спайков                                                                                            |
| <code>Amplitudes_Heights.csv</code> | Для каждой записи: количество волн, массив индивидуальных амплитуд и высот в виде строк через «;»                                                                                              |

Все три файла сохраняются в папке `Output/` и не удаляются при очистке кеша. Из финального окна «Pipeline Complete» доступен экспорт в форматах **.txt** (табуляция) и **.mat** (MATLAB). Файл `Amplitudes_Heights.csv` при экспорте в `.mat` преобразуется в cell-массив: каждый элемент — вектор переменной длины, соответствующий одной 20-минутной записи.

## 2. Пошаговое руководство пользователя

В этой главе описана работа с программой от запуска до получения итоговых файлов. Каждый раздел соответствует одному шагу конвейера. Для каждого шага показано, что нужно сделать, что отображается на экране и какой файл сохраняется в результате.

### 2.1 Подготовка к работе

#### 2.1.1 Системные требования

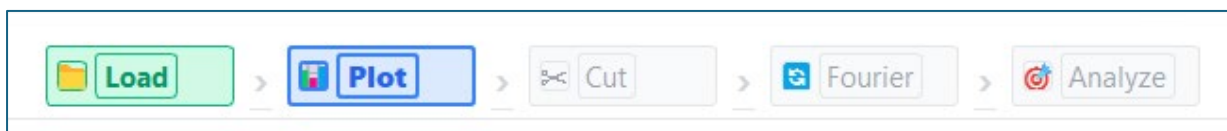
Программа работает на Windows 10/11.

- Оперативная память: не менее 4 ГБ (рекомендуется 8 ГБ для длинных записей)
- Место на диске: объём папки Output примерно равен объёму исходных .dat файлов

#### 2.1.2 Запуск программы

Запустить файл `Data_Preprocessing_Pipeline.exe`

Откроется главное окно **Шаг 1**. В верхней части окна всегда отображается индикатор текущего шага конвейера:



[ Рис. 2 — Индикатор прогресса — активный шаг выделен синим, пройденные - зелёным ]

#### 2.1.3 Папка Output и система checkpoint'ов

Все промежуточные и финальные файлы сохраняются в папке `Output/` рядом с программой. При каждом запуске программа **автоматически проверяет** какие файлы уже существуют и предлагает продолжить с соответствующего шага, пропустив уже выполненные.

## 2.2 Шаг 1 — Загрузка и конвертация файлов

|        |        |           |           |        |
|--------|--------|-----------|-----------|--------|
| 1 Load | 2 Plot | 3 Fourier | 4 Analyze | ✓ Done |
|--------|--------|-----------|-----------|--------|

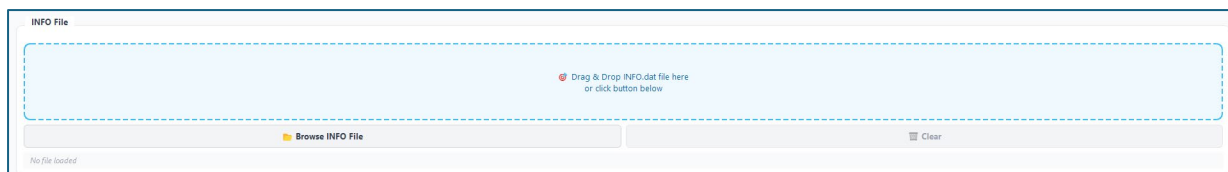
**Цель шага:** загрузить файлы с точками и метаданные, объединить их в единый временной ряд и сохранить в `Step1_TXTtoCSV.csv`.

#### 2.2.1 Добавление файлов

Добавить файлы можно двумя способами:

1. **Перетащить** файлы прямо в серую зону с пунктирной рамкой в центре окна (drag & drop).
2. **Нажать кнопку «Добавить файлы»** и выбрать файлы через стандартный диалог.





[ Рис. 3 — Окно Шага 1 — зона drag & drop и список добавленных файлов ]

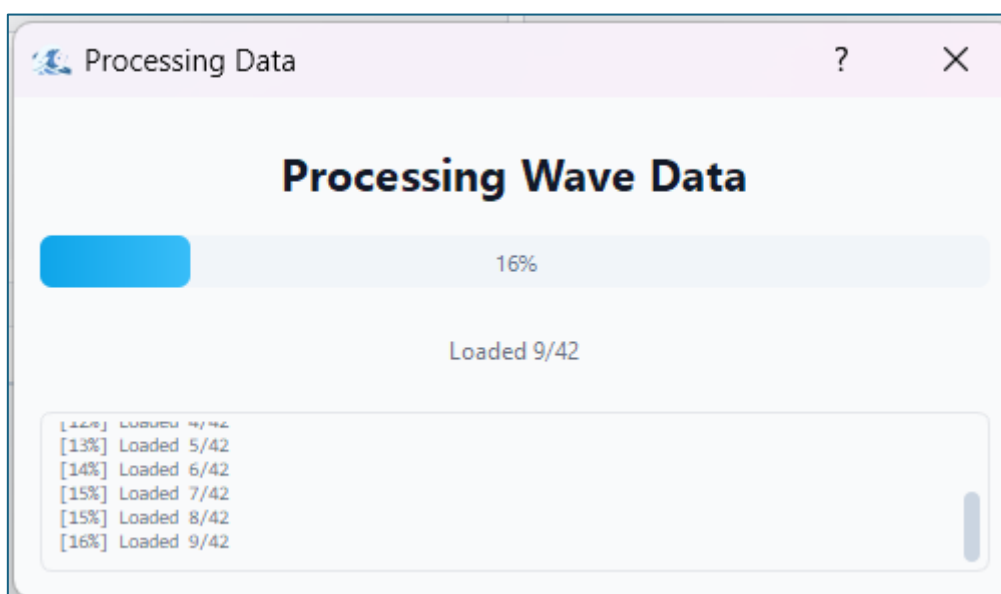
После добавления в списке отображаются все загруженные файлы. Необходимо убедиться, что присутствует **ровно один файл INFO** и один или несколько файлов **.dat**. Если файлов **.dat** несколько, они будут объединены в хронологическом порядке их имён.

**Совет:** Файлы **.dat** можно добавлять по одному или выделить все сразу в диалоге (Ctrl+A). Лишние файлы можно удалить из списка, выделив их и нажав кнопку «Удалить выбранные».

### 2.2.2 Запуск конвертации

Нажать кнопку **«Process & Convert»**. Появится прогресс-бар. Программа последовательно:

- Читает INFO файл и извлекает частоту дискретизации и время начала/конца записи
- Читает все **.dat** файлы и объединяет значения в один массив
- Генерирует временные метки, начиная с даты и времени из **.INFO**
- Разбивает ряд на 20-минутные блоки (reading\_number = 1, 2, 3, ...)
- Сохраняет результат в `Output/Step1_TXTtoCSV.csv`

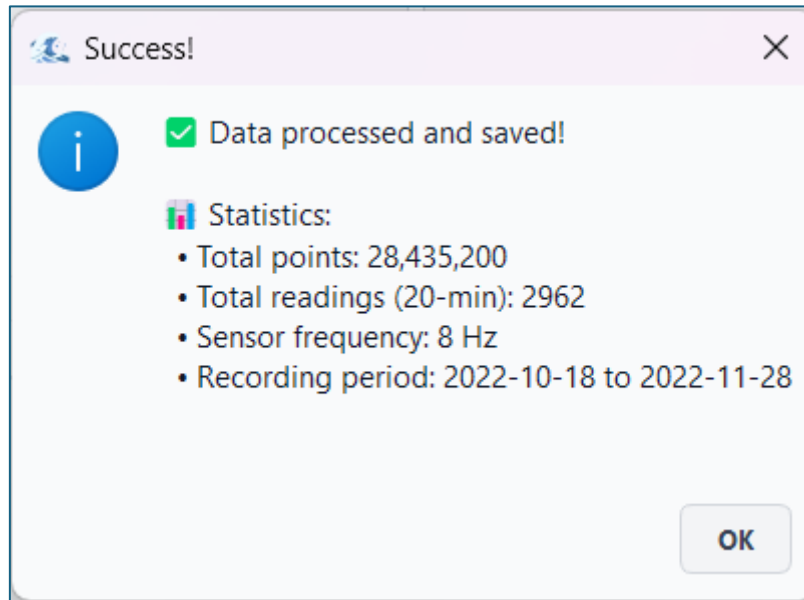


[ Рис. 4 — Прогресс-бар во время конвертации ]

### 2.2.3 Итоговое сообщение

По завершении появляется диалог с результатами:

- **Total points** — общее количество отсчётов в объединённом ряду
- **Total readings (20-min)** — количество полных 20-минутных блоков
- **Sensor frequency** — частота дискретизации, считанная из .INFO
- **Recording period** — начало и конец записи



[ Рис. 5 — Диалог с результатами после успешной конвертации ]

После закрытия диалога программа автоматически переходит к **Шагу 2**.

## 2.3 Шаг 2 — Визуализация и обрезка

|        |        |           |           |        |
|--------|--------|-----------|-----------|--------|
| 1 Load | 2 Plot | 3 Fourier | 4 Analyze | ✓ Done |
|--------|--------|-----------|-----------|--------|

**Цель шага:** визуально проверить запись, убрать «ноги» погружения и подъёма, сохранить обрезанный нуль-средний сигнал в `Step2_Zero_Mean.csv`.

### 2.3.1 Чтение графика

После загрузки данных отображается график смещения поверхности. По умолчанию строится **субсемплированная версия** (не более 10 000 точек) для быстрой отрисовки. Ось X — дата, ось Y — высота водяного столба над датчиком в метрах.



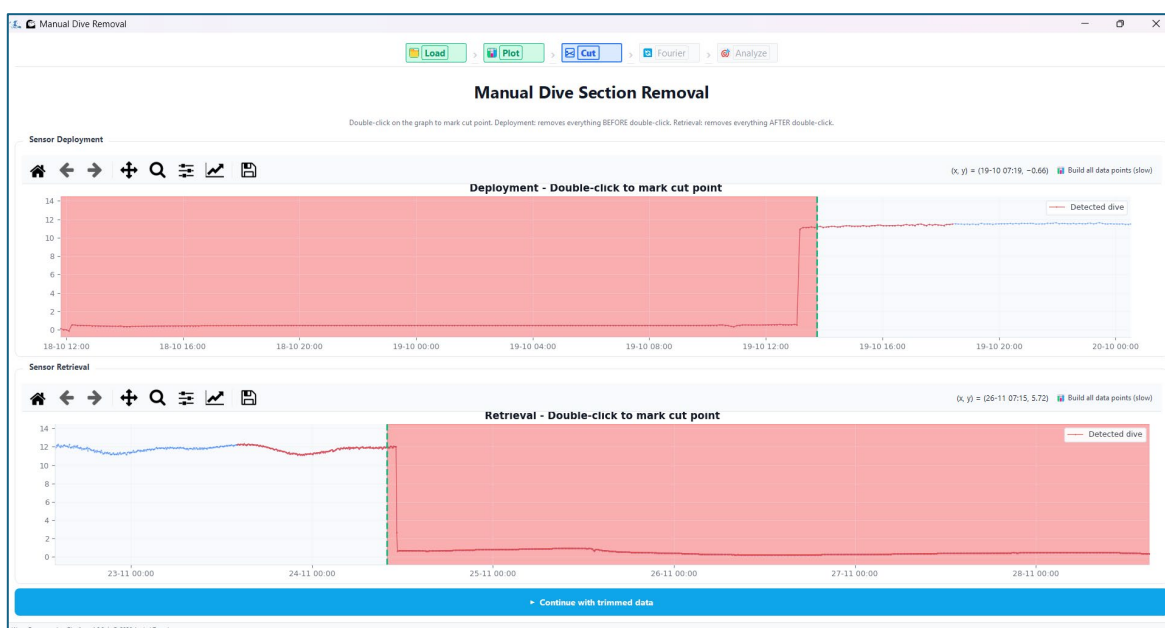
[ Рис. 6 — Главное окно Шага 2 — полная запись с выделенными «ногами» погружения ]

Участки, определённые детектором как ноги погружения или подъёма, выделены **красным цветом**. Это те части записи, когда датчик ещё двигался через водную толщу и данные не являются волновым сигналом.

## 2.3.2 Ручная обрезка

Если границы записи нужно скорректировать — нажать **«Ручная обрезка»** (кнопка активна только при наличии обнаруженных погружений).

В режиме ручной обрезки возможно отображение двух (погружение и всплытие прибора) или одного (только погружение/только всплытие) графика. На графике двойным щелчком мыши пользователь выбирает с какой (до какой) точки обрезать запись. Далее нажать **«Применить обрезку»**.



[ Рис. 7 — Режим ручной обрезки ]

**Совет:** Перед кликом по графику нажмите кнопку «Построить полностью» в панели инструментов — тогда клик будет соответствовать точному положению в данных, а не в субсемплированной версии (если важна точность).

### 2.3.3 Что происходит с данными

После подтверждения обрезки программа:

- Удаляет точки вне выбранного диапазона
- Вычитает среднее значение каждой 20-минутной записи (глубина в точке погружения, далее пойдет в расчет параметра относительной глубины).
- Сохраняет результат в `Output/Step2_Zero_Mean.csv`
- Сохраняет субсемплированный кеш в `Output/Step2_Visualization.csv`

Затем программа автоматически переходит к **Шагу 3**.

## 2.4 Шаг 3 — Фурье-фильтрация

|        |        |           |           |        |
|--------|--------|-----------|-----------|--------|
| 1 Load | 2 Plot | 3 Fourier | 4 Analyze | ✓ Done |
|--------|--------|-----------|-----------|--------|

**Цель шага:** удалить из сигнала долгопериодные компоненты (приливы, дрейф), сохранить только ветровые волны в `Step3_Transformed.csv`.

### 2.4.1 Вычисление спектра

При входе в Шаг 3 программа автоматически вычисляет полный спектр Фурье всего сигнала. Это может занять несколько секунд для длинных записей. По окончании открывается **окно спектра с двумя графиками**:



[ Рис. 8 — Окно Шага 3 — два графика спектра и кнопка Apply ]

- **Верхний график** — диапазон угловых частот от 0.1 рад/с и выше. Здесь видны пики ветровых волн. Горизонтальная шкала линейная.

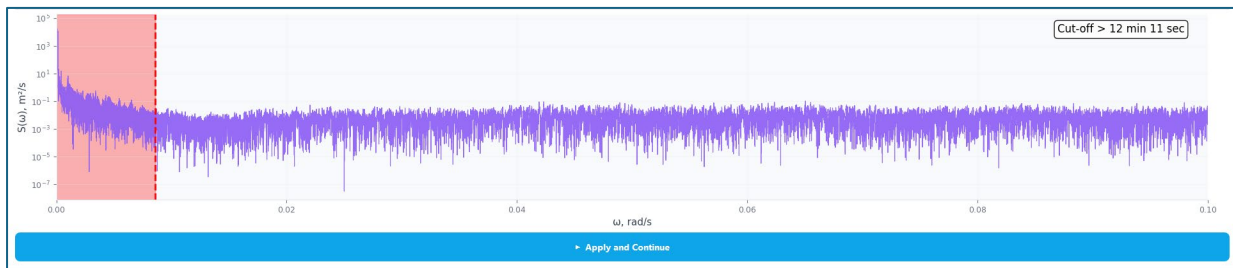
- **Нижний график** — диапазон 0–0.1 рад/с в логарифмическом масштабе по вертикали. Здесь видны приливы и инфрагравитационные волны с очень малыми угловыми частотами. Здесь же можно построить полный спектр и увидеть «хвост» в логарифмическом масштабе.

Дополнительно в верхней части окна можно построить **спектрограмму** (оконное преобразование Фурье). Пользователь выбирает длину окна (в минутах, по умолчанию 10), шаг окна (в секундах, по умолчанию 60) и процент спектра на спектрограмме (20% по умолчанию — соответствуют 0,2 \* максимальная частота в спектре).

## 2.4.2 Выбор частоты среза

Двойным кликом по **нижнему графику** выбирается частота среза  $\omega_c$ . После клика:

- На обоих графиках появляется вертикальная красная линия на выбранной частоте
- Над нижним графиком отображается соответствующий период:  $T = 2\pi / \omega_c$  (в секундах и минутах)
- Область левее линии закрашивается — это компоненты, которые будут удалены из реальной и мнимой (зеркально) части спектра.
- Активируется кнопка **«Apply & Continue»**



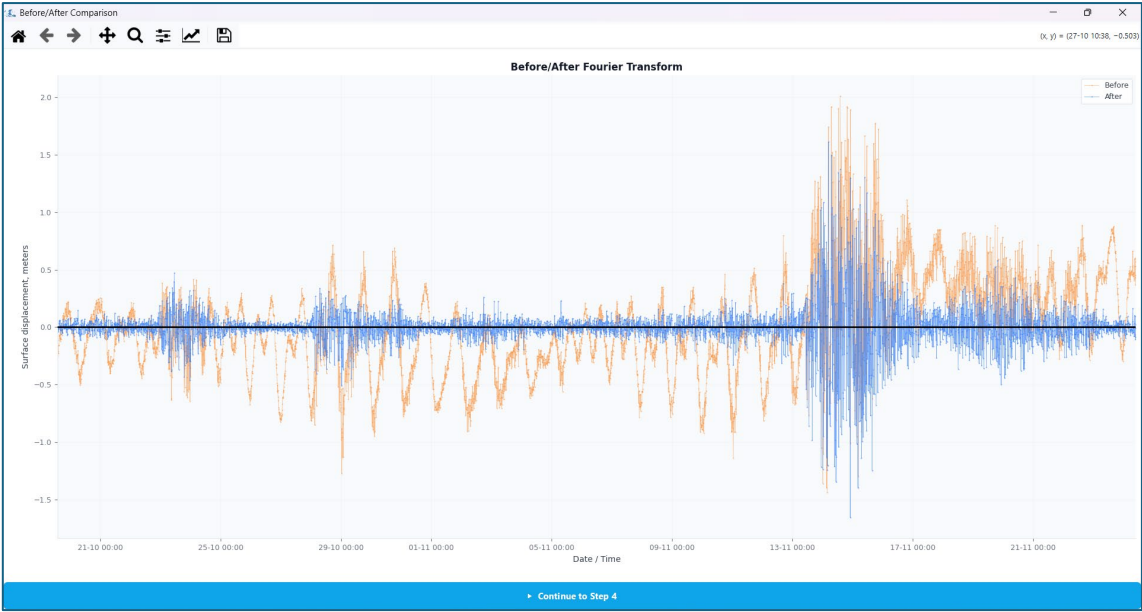
[ Рис. 9 — Нижний спектр с выбранной частотой среза — красная вертикальная линия и закрашенная зона ]

## 2.4.3 Применение фильтра и окно сравнения

Нажать **«Apply & Continue»**. Программа:

1. Добавляет зеркальные хвосты к сигналу для подавления краевых артефактов
2. Применяет БПФ к расширенному сигналу
3. Обнуляет компоненты ниже выбранной частоты
4. Выполняет обратное БПФ
5. Отрезает хвосты — возвращает сигнал исходной длины
6. Сохраняет результат в `Step3_Transformed.csv`

Затем автоматически открывается **окно сравнения Before/After**. На одном графике наложены два сигнала: **оранжевый** — исходный до фильтрации, **синий** — отфильтрованный (оба субсемплированные для ускорения работы программы).



[ Рис. 10 — Окно Before/After — оранжевый исходный и синий отфильтрованный сигнал ]

После закрытия окна сравнения программа переходит к **Шагу 4**.

## 2.5 Шаг 4 — Анализ и вычисление параметров

|        |        |           |           |        |
|--------|--------|-----------|-----------|--------|
| 1 Load | 2 Plot | 3 Fourier | 4 Analyze | ✓ Done |
|--------|--------|-----------|-----------|--------|

**Цель шага:** пройти по всем 20-минутным записям, применить опциональную очистку и рассчитать полный набор волновых параметров.

### 2.5.1 Настройки перед запуском

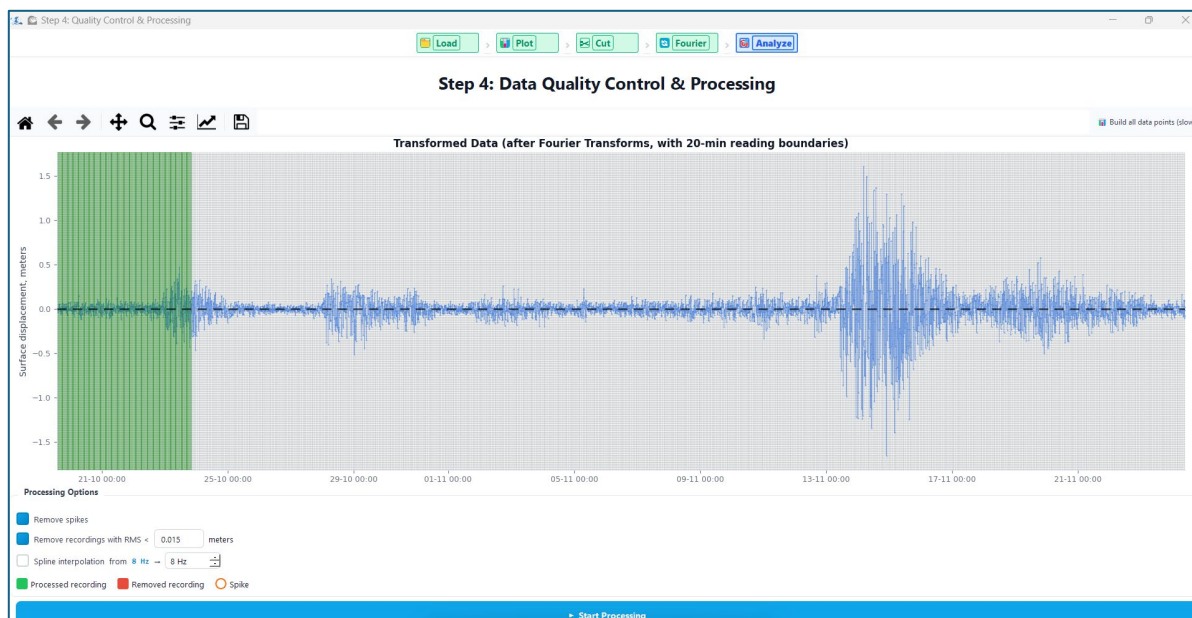
Перед нажатием кнопки Start Processing можно настроить три опции:

| Опция                        | Параметр  | Описание                                                                                                                                                                                        |
|------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Remove recordings with RMS < | 0.015 м   | Исключает 20-минутные записи, в которых амплитуда волнения ниже порога — «тихие» периоды без волн. Порог RMS задаётся вручную в поле рядом.                                                     |
| Remove spikes                | —         | Находит и исправляет отдельные аномальные точки (скачки > 6σ) путём линейной интерполяции между соседями.                                                                                       |
| Spline interpolation         | target Hz | Пересемплирует сигнал на более высокую частоту кубическим сплайном перед вычислением амплитуд. Полезно для датчиков с низкой частотой (1 Гц) — позволяет точнее оценить амплитуду каждой волны. |

### 2.5.2 Процесс обработки в реальном времени

Нажать **«Start Processing»**. Откроется прогресс-бар, затем начнёт строиться **граф в реальном времени**. Каждая 20-минутная запись обрабатывается последовательно и отображается по мере вычисления:

- **Зелёные участки** — записи, прошедшие RMS-фильтр и включённые в расчёт параметров
- **Красные участки (полупрозрачные)** — записи, исключённые как «тихие» (RMS ниже порога)
- **Оранжевые кружки** — исправленные спайки (если включено удаление спайков)

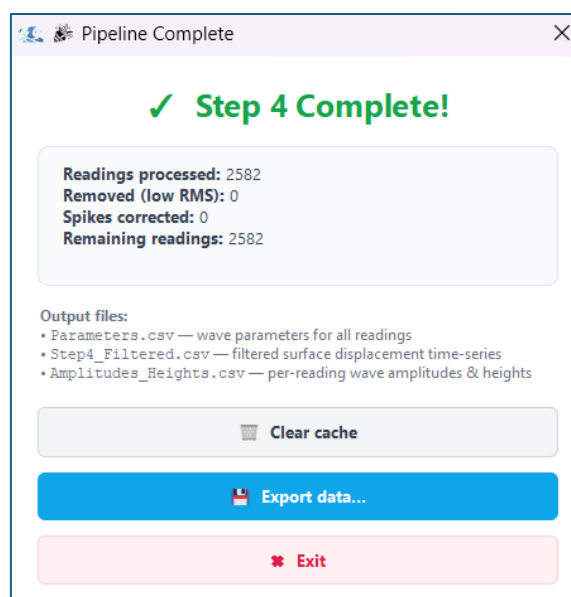


[ Рис. 12 — График Шага 4 в реальном времени ]

После обработки последней записи автоматически открывается окно **Pipeline Complete**.

### 2.5.3 Окно Pipeline Complete

Окно показывает итоговую статистику обработки:



[ Рис. 13 — Окно Pipeline Complete — статистика и список выходных файлов ]

Из этого окна доступна кнопка **«Export»** для сохранения результатов в нужном формате.



## 2.6 Экспорт результатов

Кнопка «**Export**» в окне Pipeline Complete запускает диалог выбора формата. Доступны два формата:

- **.txt (tab-separated)** — текстовый файл с табуляцией в качестве разделителя. Открывается в Excel, MATLAB, Python, любом текстовом редакторе.
- **.mat (MATLAB)** — бинарный файл формата MATLAB v5. Содержит те же данные в виде матриц и cell-массивов. Напрямую загружается командой `load('filename.mat')` в MATLAB.

После выбора формата открывается диалог выбора папки назначения. Экспортируются три файла:

| Файл               | Содержимое                                                                                                                                                               |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parameters         | Таблица: одна строка = одна 20-минутная запись. Колонки — все вычисленные параметры (RMS, Hs, Tz, kh, BFI и др.). Подробнее — Глава 3.                                   |
| Step4_Filtered     | Временной ряд смещения поверхности после удаления тихих записей и исправления спайков. Колонки: timestamp, surface_displacement, reading_number.                         |
| Amplitudes_Heights | Для каждой записи: n_waves — число волн, amplitudes — массив индивидуальных амплитуд, heights — массив высот. В .mat формате хранятся как cell-массивы переменной длины. |

[ Рис. 15 — Диалог выбора формата экспорта ]

### 2.6.1 Загрузка результатов в MATLAB

Все три .mat файла загружаются стандартной командой `load`. Ниже приведены примеры для каждого файла с пояснениями.

**Parameters.mat** — таблица волновых параметров. Каждая переменная — это вектор-столбец длиной N (число 20-минутных записей):

```
data = load('Parameters.mat');
reading = data.reading_number; % номера записей, вектор Nx1
Hs      = data.Hs;             % значительная высота, м
Hs13    = data.Hs_1_3;         % H 1/3 из инд. волн, м
Tz      = data.Tz;             % период нуль-пересечений, с
BFI     = data.BFI_proper;     % индекс Бенджамина-Фейра
rho     = data.rho;            % параметр регулярности
```

**Step4\_Filtered.mat** — временной ряд после очистки. Содержит три переменные:

```
ts = load('Step4_Filtered.mat');
eta  = ts.surface_displacement; % смещение поверхности, м
reading = ts.reading_number;    % номер 20-мин записи
% Время хранится как дни от 0000-01-01 00:00:00
```



```
% Для перевода в стандартный MATLAB datenum (от 0000-01-00):
t_dn    = ts.timestamp + 1;          % поправка на 1 день
t_dt    = datetime(t_dn, 'ConvertFrom', 'datenum');
% Пример: построить временной ряд первой записи
mask = reading == 1;
plot(t_dt(mask), eta(mask));
```

**Amplitudes\_Heights.mat** — индивидуальные амплитуды и высоты волн. Переменные `amplitudes` и `heights` хранятся как **cell-массивы** — каждая ячейка содержит вектор переменной длины для одной записи:

```
ah = load('Amplitudes_Heights.mat');
n_waves = ah.n_waves;          % число волн в каждой записи
% Амплитуды и высоты первой записи:
A1 = ah.amplitudes{1};         % вектор амплитуд, м
H1 = ah.heights{1};            % вектор высот волн, м
% Гистограмма высот волн по всей записи:
all_H = vertcat(ah.heights{:});
histogram(all_H, 30);
```

**⚠ Примечание:** Переменная `timestamp` в `Step4_Filtered.mat` хранится как количество дней от 0000-01-01. Стандартный MATLAB `datenum` считает от 0000-01-00 (на 1 день раньше), поэтому необходимо добавить поправку +1 перед конвертацией через `datetime(..., 'ConvertFrom', 'datenum')`.

## 2.6.2 Загрузка результатов в Python

```
import pandas as pd, scipy.io as sio, numpy as np
params = pd.read_csv('Parameters.txt', sep='\t')
mat = sio.loadmat('Amplitudes_Heights.mat')
heights_r1 = mat['heights'][0, 0].flatten()
```